Media Streaming

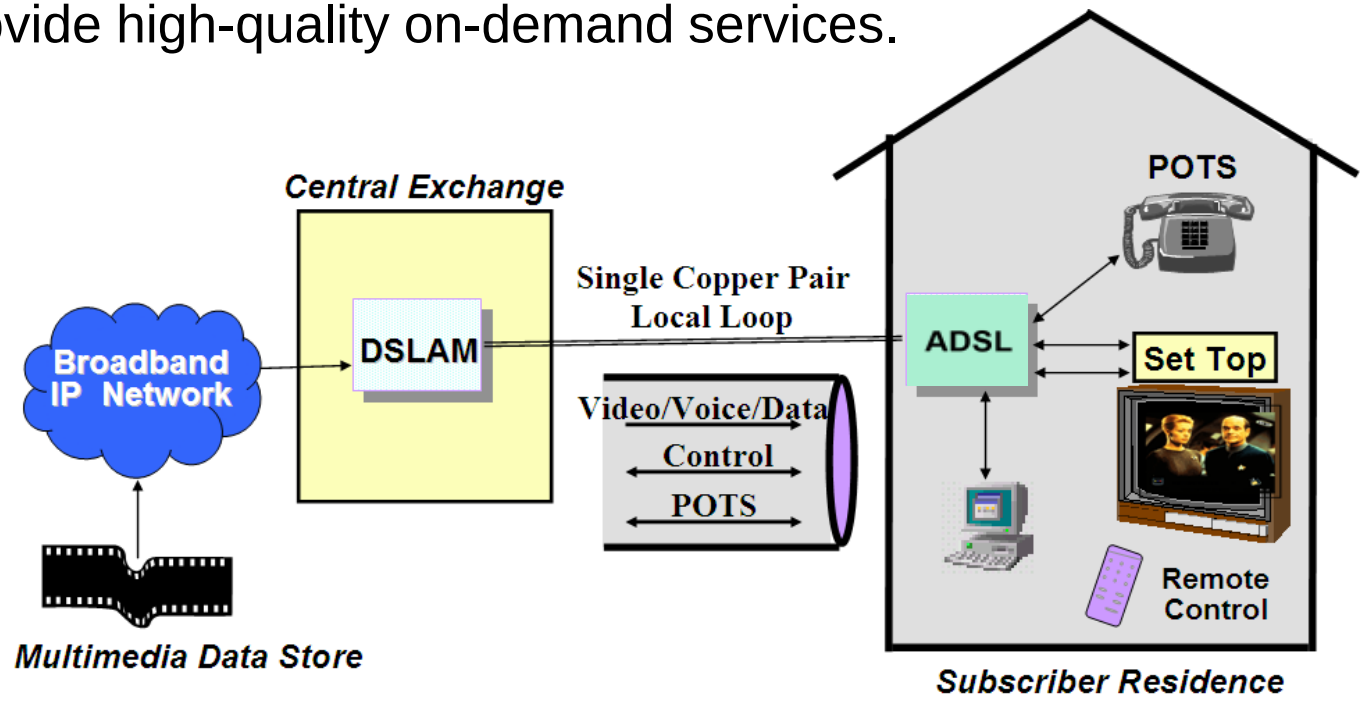
- IPTV v.s. Internet TV
- Media Transport
- Streaming Protocol
- Streaming Server Architecture

IPTV v.s. Internet TV

- **What's IPTV?**
  - **historically there have been many different definitions of "IPTV"**
    - Elementary streams over IP networks,
    - Transport streams over IP networks,
    - and a number of proprietary systems
  - **The official definition approved by the "ITU focus group on IPTV" :**
    - *"IPTV is defined as multimedia services such as television/video/audio/text/graphics/data delivered over IP based networks managed to provide the guaranteed quality of service and experience, security, interactivity and reliability."*
- **What's Internet TV?**
  - Contrast to IPTV, service provider has no control over the final delivery and so broadcasts on a "best effort" basis.
  - Elementary streams over IP networks and proprietary variants used such as YouTube are rarely considered to be IPTV services

IPTV v.s. Internet TV

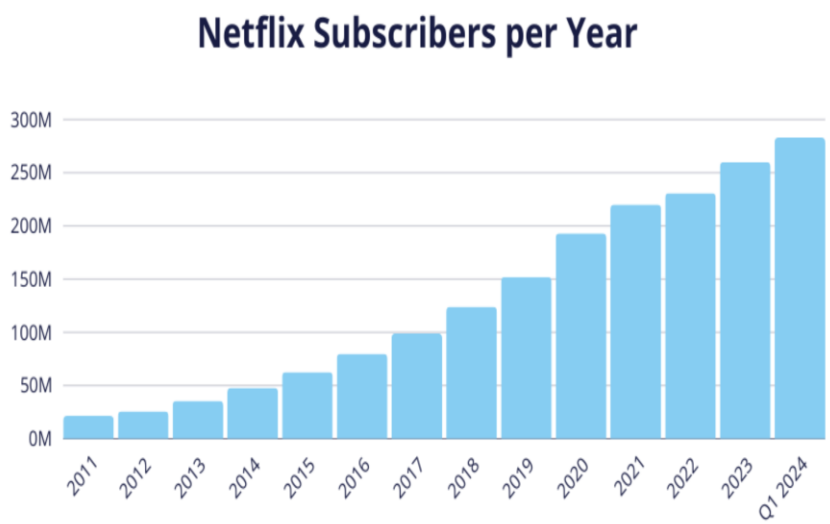
- **IPTV Example – CHT MOD**
  - Provide IPTV services (not free lunch culture as Internet) to both business and residential customers via TV.
    - VOD (video on-demand): unicast
    - Near VOD: time-staggered multicast
  - Using broadband IP network with ADSL access. technology to provide high-quality on-demand services.



IPTV v.s. Internet TV

Internet TV example – Netflix

- 1997 - Founded (DVD rental service)
- 2007 - Launched as a streaming service provider
- … - Gradual expansion to many markets
- 2010 - Began to roll out internationally
- 2024 - Global paid subscriber reached 301.6M over 190 countries worldwide



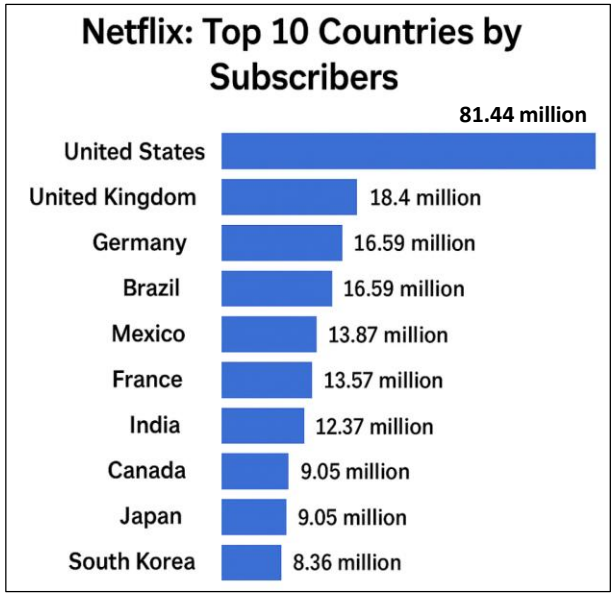


Figure source：Statista

IPTV v.s. Internet TV

Netflix

影音串流服務: 使用者上網訂閱，即可以4K的高畫質在能上網的裝置上即時觀賞其精選的熱門電視節目及電影。

成功的原因

- Anytime：**一次看完**
  - 2013年熱門影集《紙牌屋》一上線，Netflix就把第一季、共13集的內容通通上線。而這部影集走紅，吸引更多人訂閱。據 MarketingCharts報導，62%觀眾會一次看多集影集。
- Anyone：**個人推薦準度高**
  - 點擊、播放、快轉、暫停，導演、演員、影片類型等，都會被列入資料庫，結合大數據及資料探勘技術，建立個人化推薦引擎。據 Accenture資料，89%的觀眾曾透過網路收看長篇影音。
- Anywhere：**跨裝置隨選隨播**
  - 無論在哪裡，只要連上網路都能收看。用戶開啟影片時，Netflix會立刻找出最適合該裝置的影片來源，確保觀看品質。 據 Accenture資料，89%的觀眾曾透過網路，收看長篇影音內容。
- Content: **自製內容再造傳統影集價值**
  - 舊影集翻拍，再造傳統影集價值。例如《紙牌屋》改編自1990年代英國BBC劇集，由Netflix拍攝。

Media Streaming

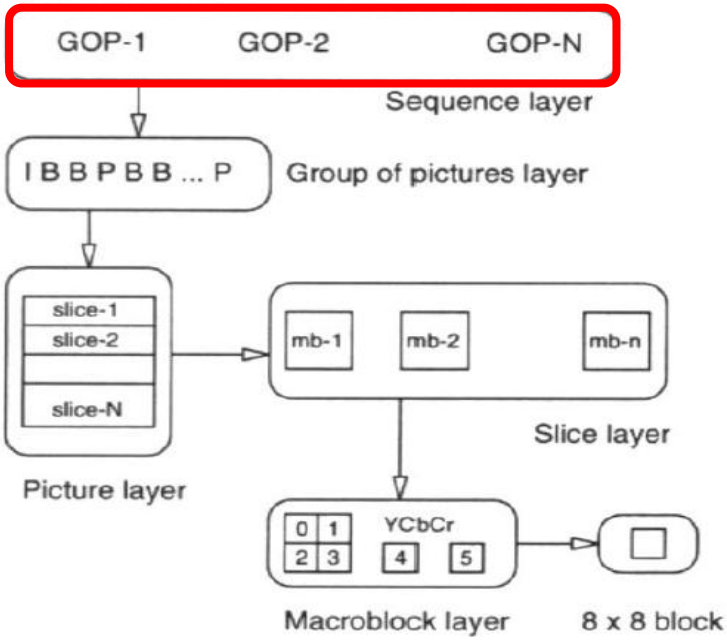
- IPTV v.s. Internet TV
- Media Transport
  - ES, PES, PS, TS
- Streaming Protocol
  - DVB IPTV
  - HTTP Live Streaming (HLS)
  - MPEG-DASH
- Streaming Server Architecture

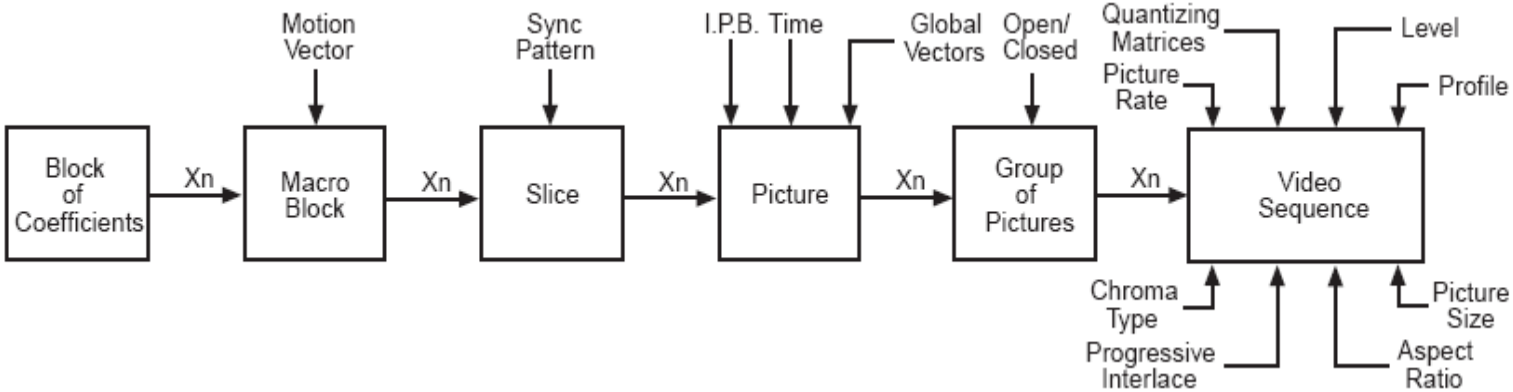
MPEG - ES

- **Elementary Stream (ES)**
  - **An ES contains only one kind of data (audio or video)**
    - The output of a video encoder is called a video ES
    - The output of a audio encoder is called an audio ES
    - It is hard to multiplex multiple ES directly into one file.

MPEG - ES

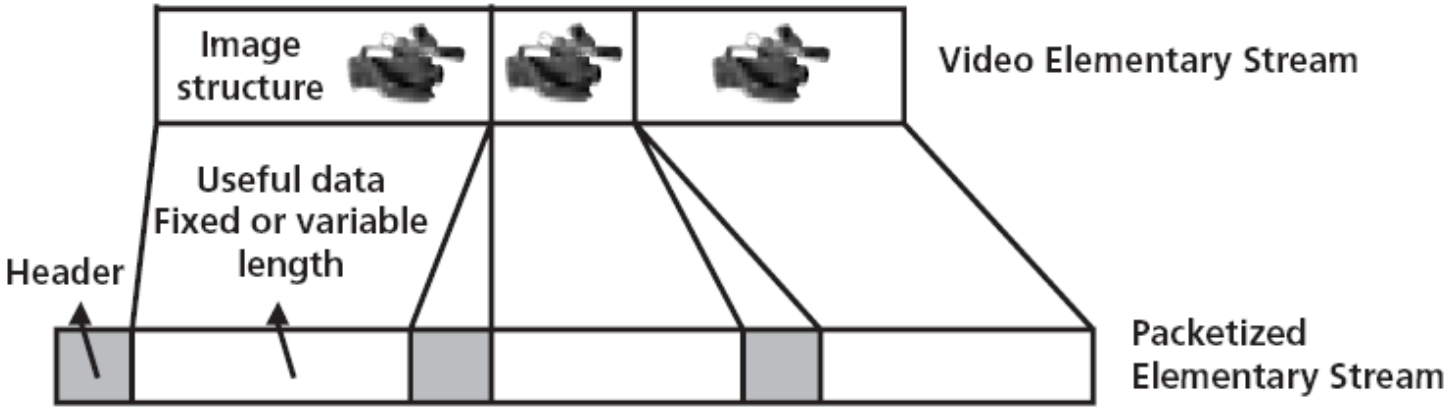
- **Ex. Video Elementary Stream (MPEG2)**





MPEG - PES

- **Packetized Elementary Stream (PES)**
  - For practical purposes, the continuous elementary streams carrying audio or video need to be broken into ***packets***, called **PES**.
  - These PESs are identified by headers containing ***time stamps*** for
    1. ***Time-base correction*** in each elementary stream.
    2. ***Synchronizing*** video and audio elementary streams to make a program.

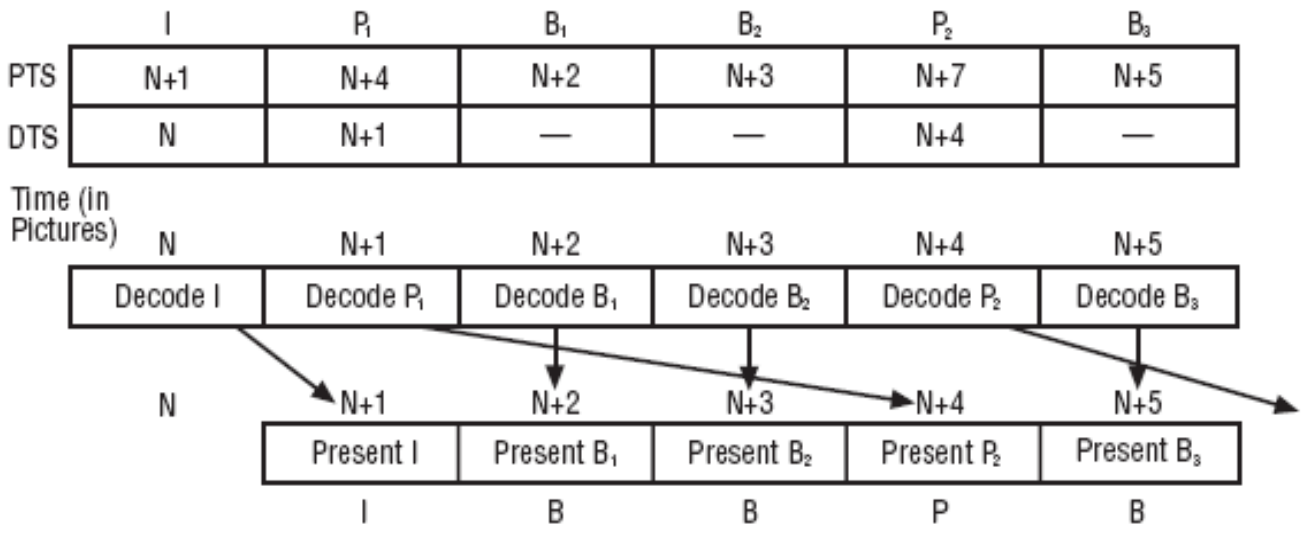


MPEG - PES

- **PTS and DTS**
  - **Time Stamp:** a time stamp is a 33-bit number that is a sample of a counter driven by a *90-kHz* clock.
  - pictures can be presented in the order IBBP, but will be transmitted in the order IPBB. Consequently, two types of time stamp exist.
    - **DTS (Decode time stamp)** indicates when a picture must be *decoded*.
    - **PTS (Presentation time stamp)** indicates when it must be presented.
  - **Lip sync** is obtained by incorporating time stamps into the headers in both video and audio PES packets.
    - Because audio packets are never transmitted out of sequence, there is ***no DTS in an audio packet***.

MPEG - PES

- **Example of DTS and PTS**
  - I picture has DTS and PTS, separated by one picture period.
  - P picture also has DTS and PTS, but the separation is three picture periods to allow for the intervening B pictures.
  - B are decoded and presented simultaneously so only has PTS.
  - Decoding sequence IPBB -> presentation sequence IBBP



MPEG - PS

- **Program Stream (PS)**
  - PS results from combining one or more *PES streams with "common" time base* into a single stream using TDM.
    - PS packets can be of ***variable*** *and* ***relatively-great*** length.
  - PS is used in *error-free* data transfer environment, like DVD
    - error-free → Using large packet length
    - Storage medium → allowing variable rate → using variable length
  - VBR (variable bit-rate)
    - For a given quality, the data rate of compressed video varies with picture type and content. A VBR channel will give best results.

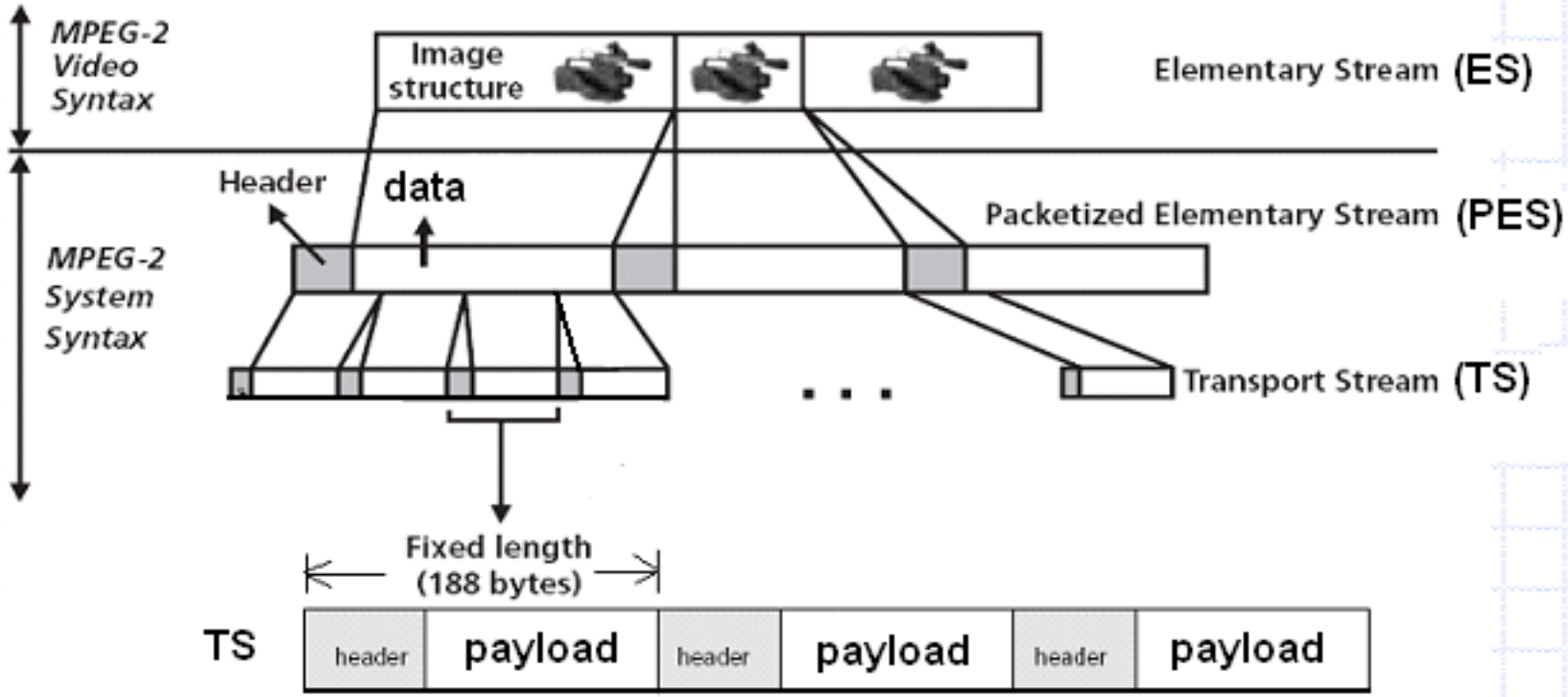
- Storage medium: can be slowed down, speeded up or physically change data transfer rate, allowing a *variable rate* to be obtained. → *PS*
- Transmission: most channels are fixed and *bit-rate is kept constant* by using stuffing (meaningless data, waste bandwidth). → TS

MPEG - TS

- **Transport Stream (TS)**
  - TS combines one or more *programs with "independent" time bases* into a single stream. *PES packets made up of elementary streams that form a program share a common time-base.*
  - TS differs from PS in that PES packets are further subdivided into *short fixed-size* packets. TS packets are fixed 188 bytes in length.
  - TS is designed for *error-prone* environment, like *broadcasting*.
  - Program multiplexing: To broadcast several programs in the same physical channel, their associated TS packets are multiplexed into a single transport stream.
  - Program information: TS also carries meta data describing the bit stream. This includes Program Association Table (PAT), Program Map Table (PMT), etc.

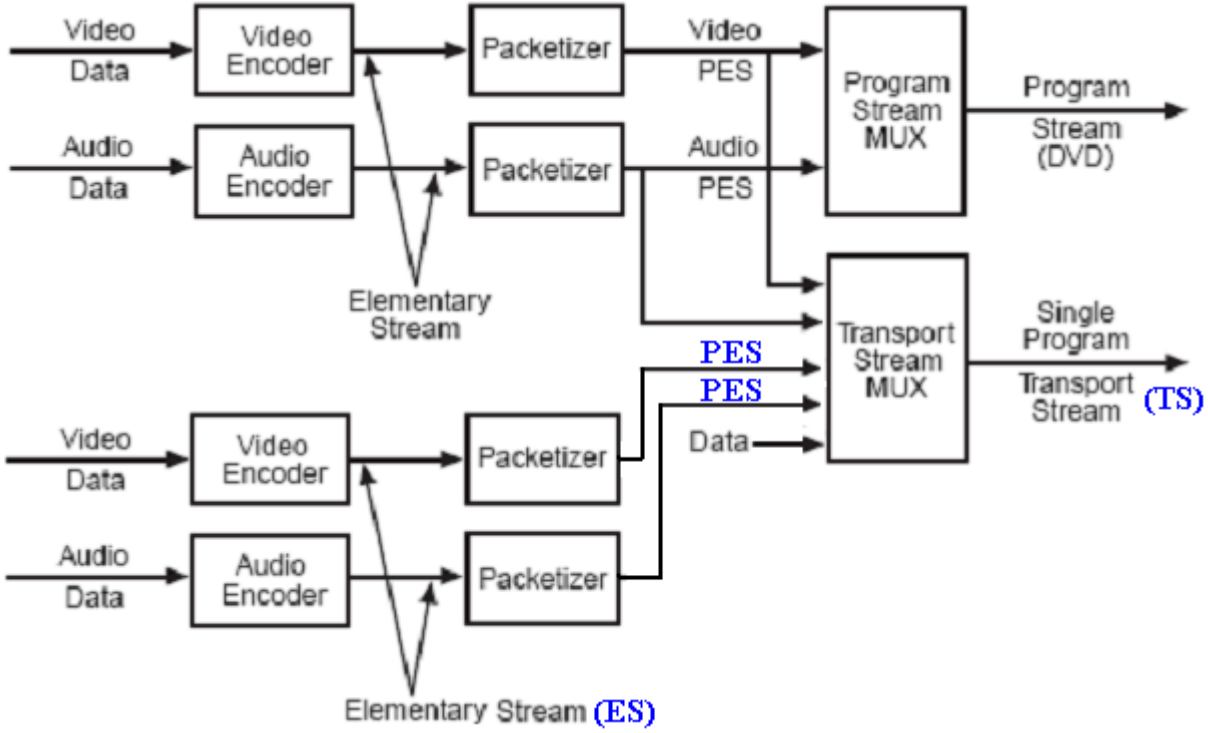
MPEG - TS

- **Stream Hierarchy (ES, PES, TS)**



MPEG - TS

- **PS & TS**



<u>Program Stream:</u> it works well on a single program with variable bit-rate in a recording environment;

<u>Transport Stream:</u> it works well on multiple programs in a fixed bit-rate transmission environment.

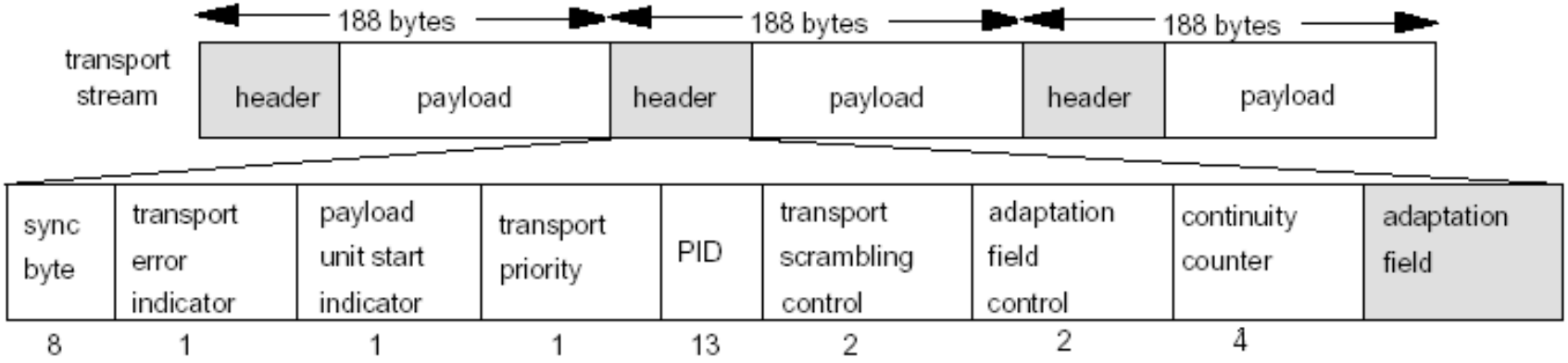
MPEG - TS

- **TS Header Format**

Syntax	No. of bits	Mnemonic
transport_packet(){		
sync_byte	**8**	**bslbf**
transport_error_indicator	**1**	**bslbf**
payload_unit_start_indicator	**1**	**bslbf**
transport_priority	**1**	**bslbf**
PID	**13**	**uimsbf**
transport_scrambling_control	**2**	**bslbf**
adaptation_field_control	**2**	**bslbf**
continuity_counter	**4**	**uimsbf**
if(adaptation_field_control=='10'		
‖ adaptation_field_control=='11'){		
adaptation_field()		
}		
if(adaptation_field_control=='01'		
‖ adaptation_field_control=='11') {		
for (i=0;i<N;i++){		
data_byte	**8**	**bslbf**
}		
}		
}		

MPEG - TS

- **TS Header Format**
  - **sync_byte**
    - a fixed 8 bit field whose value is 0x47 (or 0xb8)
  - **transport_error_indicator**
    - when set to '1' means that at least 1 uncorrectable bit error exists.

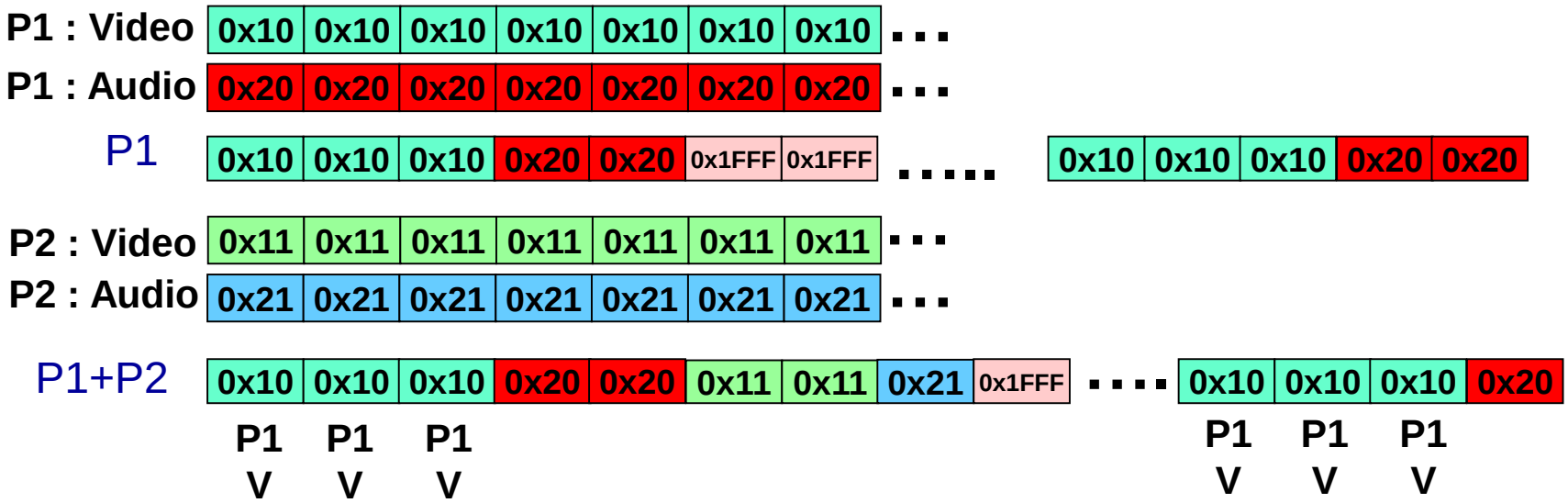


MPEG - TS

- **TS Header Format**
  - **payload_unit_start_indicator**
    - when set to ‘1’ means this TS packet carries the first byte of a PES packet or PSI section.
  - **transport_priority**
    - when set to '1' it indicates that this packet is of greater priority than other packets having the same PID with this bit set to '1'.
  - **transport_scrambling_control**
    - This indicates the scrambling mode of the TS packet payload.
    - 00: not scrambled. 01,10,11: user defined.
  - **adaptation_field_control**
    - A 2-bit field indicates whether this TS packet header is followed by an adaptation field and/or payload.
    - 01: payload only, 10: adaptation field only, 11: payload followed by adaptation, 00: reserved.
  - **continuity_counter**
    - A 4 bit field incrementing with each TS packet with the same PID. It should wrap to 0 after its maximum value 15.

MPEG - TS

- **PID (Packet IDentification)**
  - The PID is a 13 bit field (value range 0~0x1FFF), indicating the type of the data stored in the packet payload.
  - Used to identify an ES: A demultiplexer extracts ES from TS by looking for packets identified by the same PID.
  - Null packet PID: 0x1FFF



MPEG - TS

- **PSI (Program Specific Information)**
  - PSI tables act as a table of contents for the transport stream, providing the decoder with data it needs to find each program and present it to viewer.
  - **PAT (Program Association Table)**
    - Associates Program Number and PMT PIDs
  - **PMT (Program Map Table)**
    - Specifies PID values for components of one or more programs

MPEG – TS: PSI

Syntax	No. of bits	Mnemonic
program_association_section() {		
table_id	8	uimsbf
section_syntax_indicator	1	bslbf
'0'	1	bslbf
reserved	2	bslbf
section_length	12	uimsbf
transport_stream_id	16	uimsbf
reserved	2	bslbf
version_number	5	uimsbf
current_next_indicator	1	bslbf
section_number	8	uimsbf
last_section_number	8	uimsbf
for (i=0; i<N;i++) {		
program_number	16	uimsbf
reserved	3	bslbf
if(program_number == '0') {		
network_PID	13	uimsbf
}		
else {		
program_map_PID	13	uimsbf
}		
}		
CRC_32	32	rpchof
}		

PAT

Syntax	No. of bits	Mnemonic
TS_program_map_section() {		
table_id	8	uimsbf
section_syntax_indicator	1	bslbf
'0'	1	bslbf
reserved	2	bslbf
section_length	12	uimsbf
program_number	16	uimsbf
reserved	2	bslbf
version_number	5	uimsbf
current_next_indicator	1	bslbf
section_number	8	uimsbf
last_section_number	8	uimsbf
reserved	3	bslbf
PCR_PID	13	uimsbf
reserved	4	bslbf
program_info_length	12	uimsbf
for (i=0; i<N; i++) {		
descriptor()		
}		
for (i=0;i<N1;i++) {		
stream_type	8	uimsbf
reserved	3	bslbf
elementary_PID	13	uimsnf
reserved	4	bslbf
ES_info_length	12	uimsbf
for (i=0; i<N2; i++) {		
descriptor()		
}		
}		
CRC_32	32	rpchof
}		

PMT

MPEG – TS: PSI

- **PAT**
  - PID: 0x0000, table_id: 0x00
  - One PAT per Transport Stream (TS)
- **PMT**
  - PID: assigned in PAT, table_id: 0x02
  - Multiple PMTs per TS.

Program Association Table (PAT)

table_id = 0x00

...

transport_stream_id = 0x0AA1

...

Program Number	PMT PID
0x00F1	0x0100
0x00C2	0x0440
0x00B3	0x0301

PAT

PMT at PID 0x0100

Program Number	
0x00F1	
Stream Type	**PID**
PCR	0x0303
Video	0x0303
Audio	0x0206

PMT at PID 0x0440

Program Number	
0x00C2	
Stream Type	**PID**
PCR	0x0403
Video	0x0403
Audio	0x0207

PMT at PID 0x0301

Program Number	
0x00B3	
Stream Type	**PID**
PCR	0x0503
Video	0x0503
Audio	0x0208

Media Streaming

- IPTV v.s. Internet TV
- Media Transport
  - ES, PES, PS, TS
- Streaming Protocol
  - DVB IPTV
  - HTTP Live Streaming (HLS)
  - MPEG-DASH
- Streaming Server Architecture

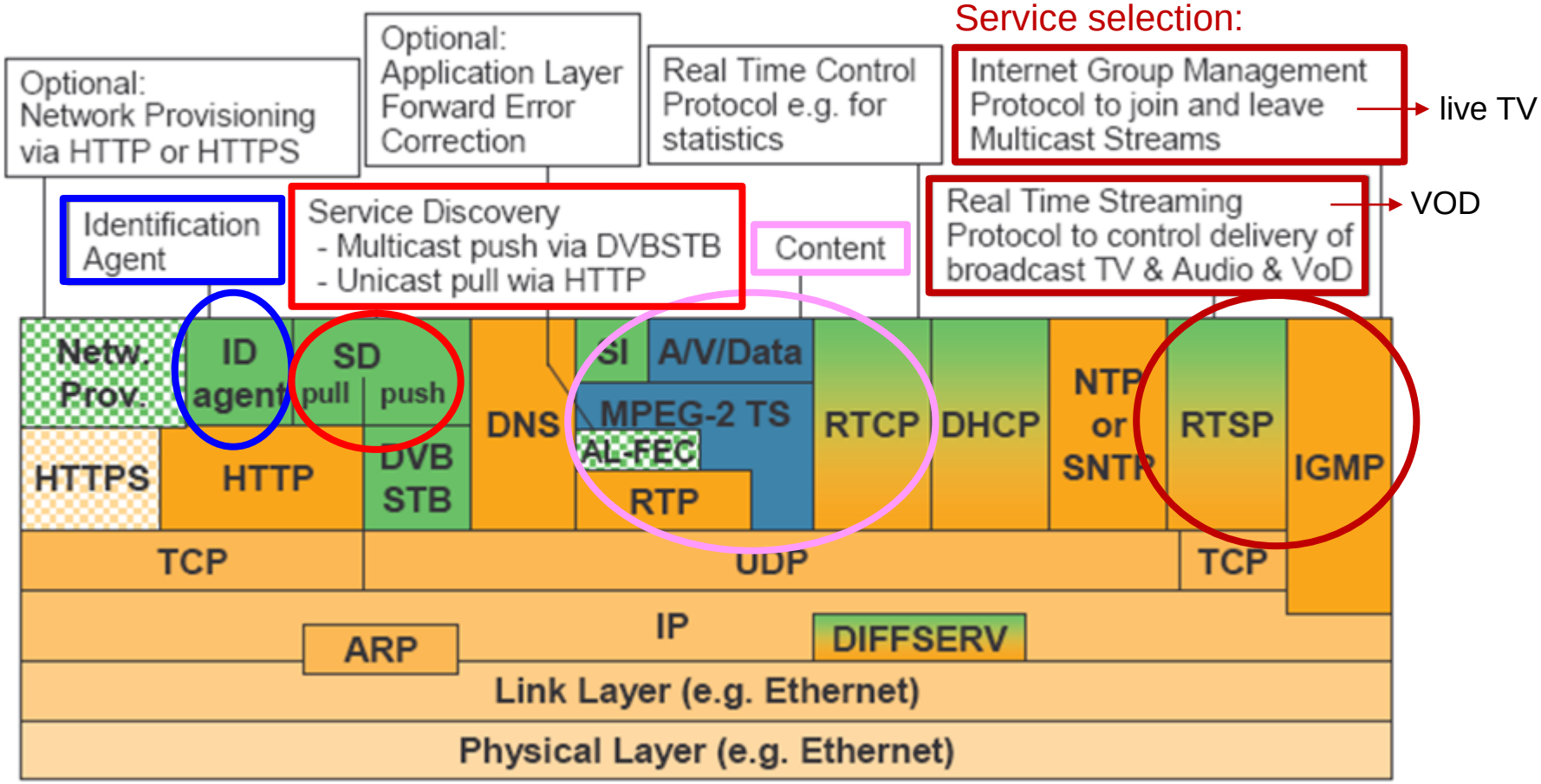
DVB IPTV Protocols

- **DVB – Subgroups with IPTV Relevance**
  - **The work inside DVB is carried out in commercial (CM) and technical (TM) subgroups**

- DVB-CM-IPTV is creating commercial requirements for delivery of broadband IPTV services to the consumer over any IP access network.
- DVB-CM-HN has created requirements for IP based DVB home networks which will be supported by future DVB-IPI technical specifications (based on DLNA guidelines).
- DVB-TM-IPI was formed to specify technologies on the interface between an IP home network and retail receivers.
- DVB-TM-TAM is defining how to add IPTV to the MHP specification. (Connection from the MHP APIs to SD&S and BCG from DVB-TM-IPI).
- DVB-TM-MUG will be defining a subset of the MHP/IPTV specification to be used for markets which are using other IPTV protocols than DVB.
- DVB-TM-CPT deals with Content Protection und Content Management, current involvement in IPTV unclear.

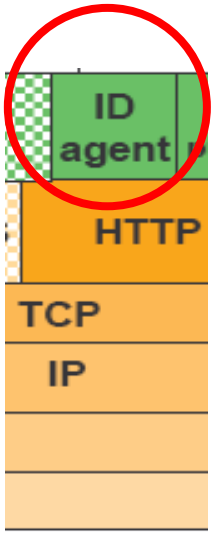
DVB IPTV Protocols

- **DVB-IPI Protocol Stack**



DVB IPTV Protocols

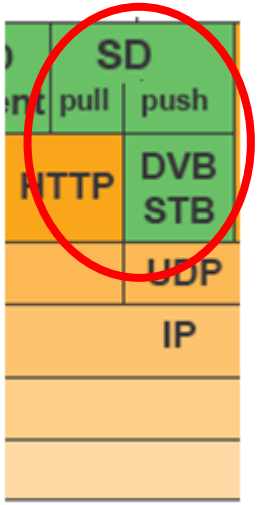
- **DVB-IPI: ID agent (Identification Agent)**
  - **Allows the service provider's system to recognize a HNED (**Home Network End Device, in DVB terms**)**
  - **After start-up the HNED is sending basic information to the service provider**
  - **A congestion avoidance mechanism shall protect the network from overload**



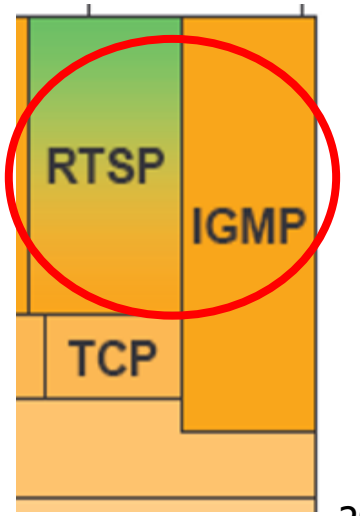
DVB IPTV Protocols

- **DVB-IPI: Service Discovery and Selection**
  - **Transport mechanism may be:**
    - Unicast (via HTTP) ⟶ pull mode
      - Client initiates requests to the server
    - Multicast (via DVBSTB) ⟶ push mode
      - Data is delivered automatically from server
  - **Service selection can be done by:**
    - IGMP: Live TV service
      - use IGMP for IPv4 connecting to a multicast stream (TV channel) and for changing from one multicast stream to another (TV channel change)
    - RTSP: VOD service
      - use RTSP to support *Internet VCR.*

Service Discovery

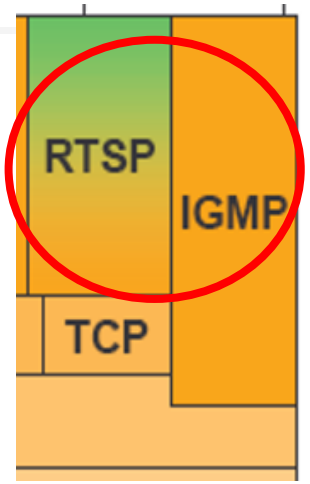


Service Selection



DVB IPTV Protocols

Service Selection

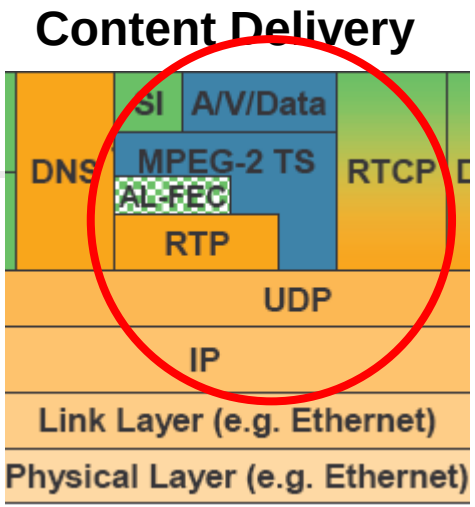


- **DVB-IPI: RTSP Client**
  - **What is RTSP (**Real-Time Streaming Protocol**)?**
    - to *control* multimedia streams delivered by *RTP*
    - The “control” includes absolute positioning within the stream recording and possible device control.

      → ***Internet VCR***.

  - RTSP protocol is too complex, only a subset is used in DVB-IPI
    - Content on demand
    - Media broadcast with trick modes (ff/rw/pause/resume)

DVB IPTV Protocols

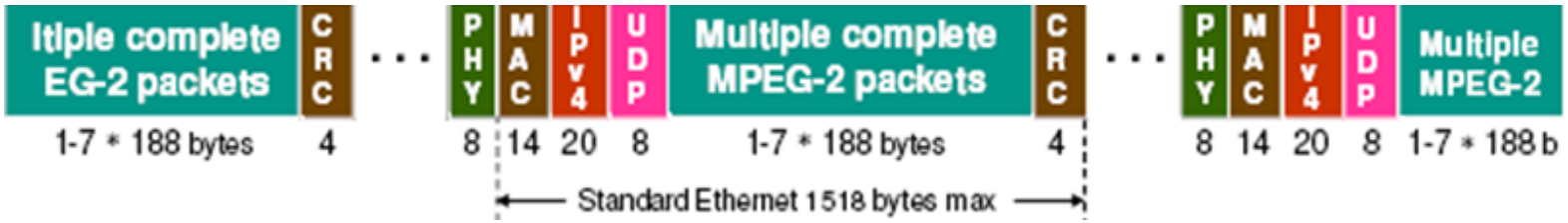


- **DVB-IPI: Delivery of MPEG-2 over IP**
  - **Two Options**
    - MPEG2 TS over UDP/IP (without RTP)
    - MPEG2 TS over RTP/UDP/IP
  - **RTP (Real-time Transport Protocol)**
    - Supports real-time applications with continuous media (e.g., audio/video)
    - The service provided by RTP include
      - Payload type identification, Sequence numbering, Time stamping.
  - **RTCP (Real-time Transport Control Protocol)**
    - Periodic transmission of control packets (no carrying data) to all participants for the synchronization of different media streams
    - as well as QoS reporting feedback from receivers
      - packets counts, fraction of lost packets, interarrival jitter, round trip time, etc.

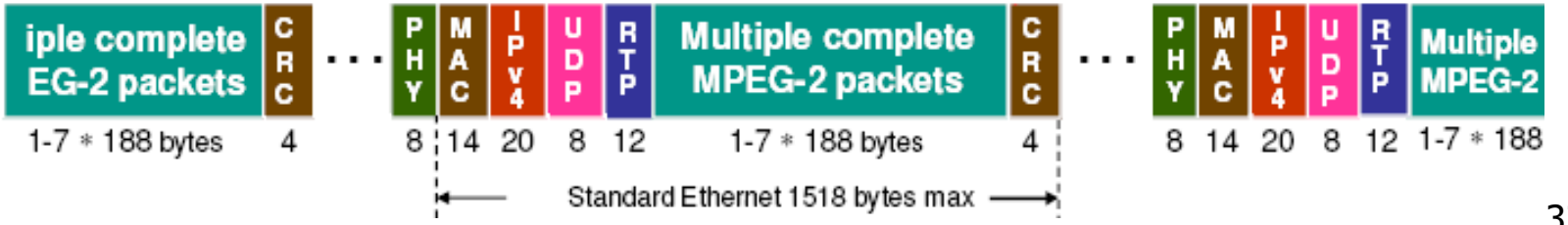
DVB IPTV Protocols

- **MPEG-2 TS video delivery**
  - **Standard Ethernet packet size: 1518 bytes (max)**
  - **Can support 1 ~ 7 MPEG2 TS packets per Ethernet frame**
    - Over **UDP/IP**: 14+20+8+188x7+4 = 1362 bytes/frame
    - Over **RTP/UDP/IP**: 14+20+8+12+188x7+4 = 1374 bytes/frame
  - **Up to 250 streams at 3.75 Mbps/stream per Giga Ethernet**

MPEG-2 TS over RTP/UDP/IP video delivery

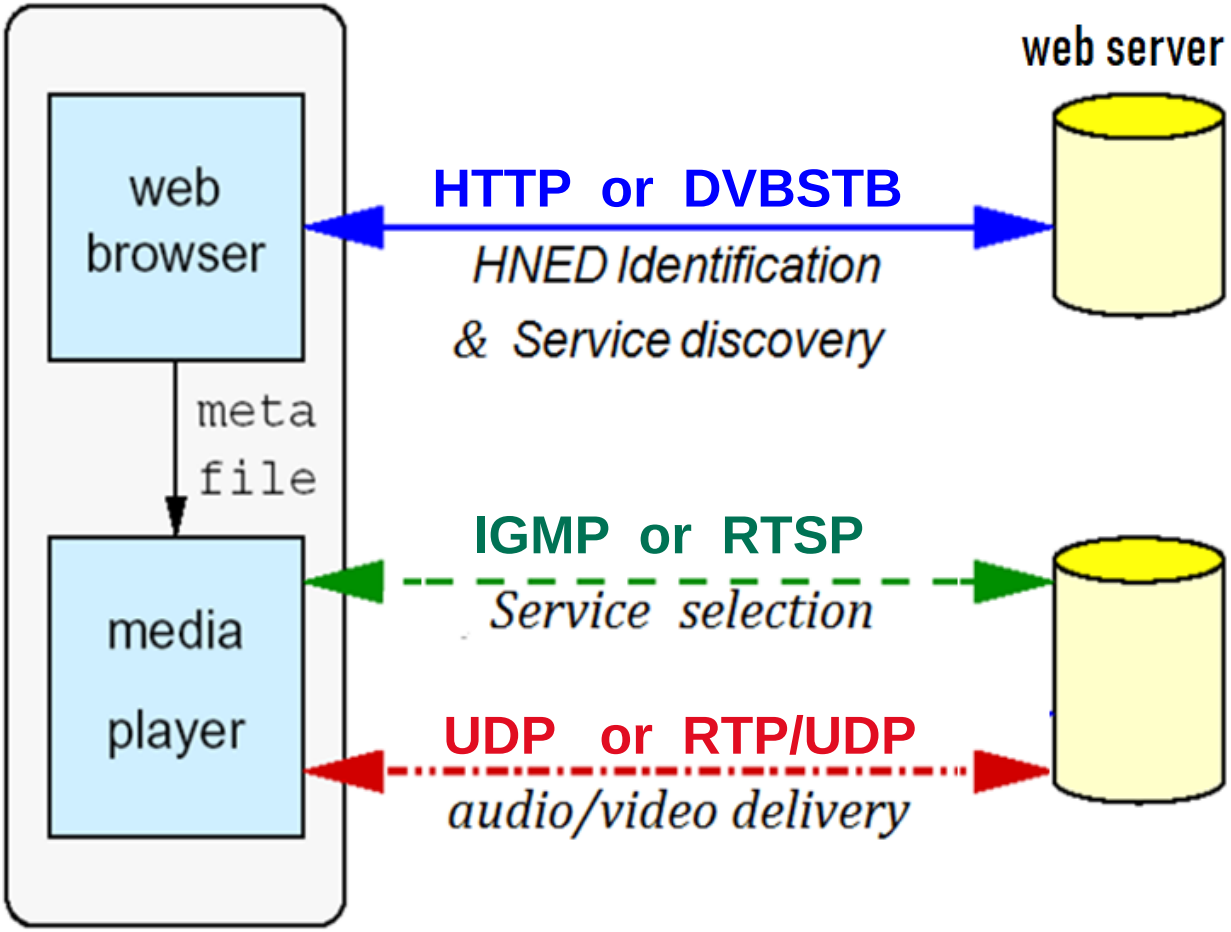


MPEG-2 TS over RTP/UDP/IP video delivery



DVB IPTV Protocols

- **DVB-IPI: An example**



ID agent
- HTTP

Service discovery
- pull: HTTP
- push: DVBSTB

Service request
- live video: IGMP
- VOD: RTSP

Service delivery
- UDP
- RTP/UDP

Media Streaming

- IPTV v.s. Internet TV
- Media Transport
  - ES, PES, PS, TS
- Streaming Protocol
  - DVB IPTV
  - HTTP Live Streaming (HLS)
  - MPEG-DASH
- Streaming Server Architecture
- P2P Streaming

HTTP Live Streaming (HLS)

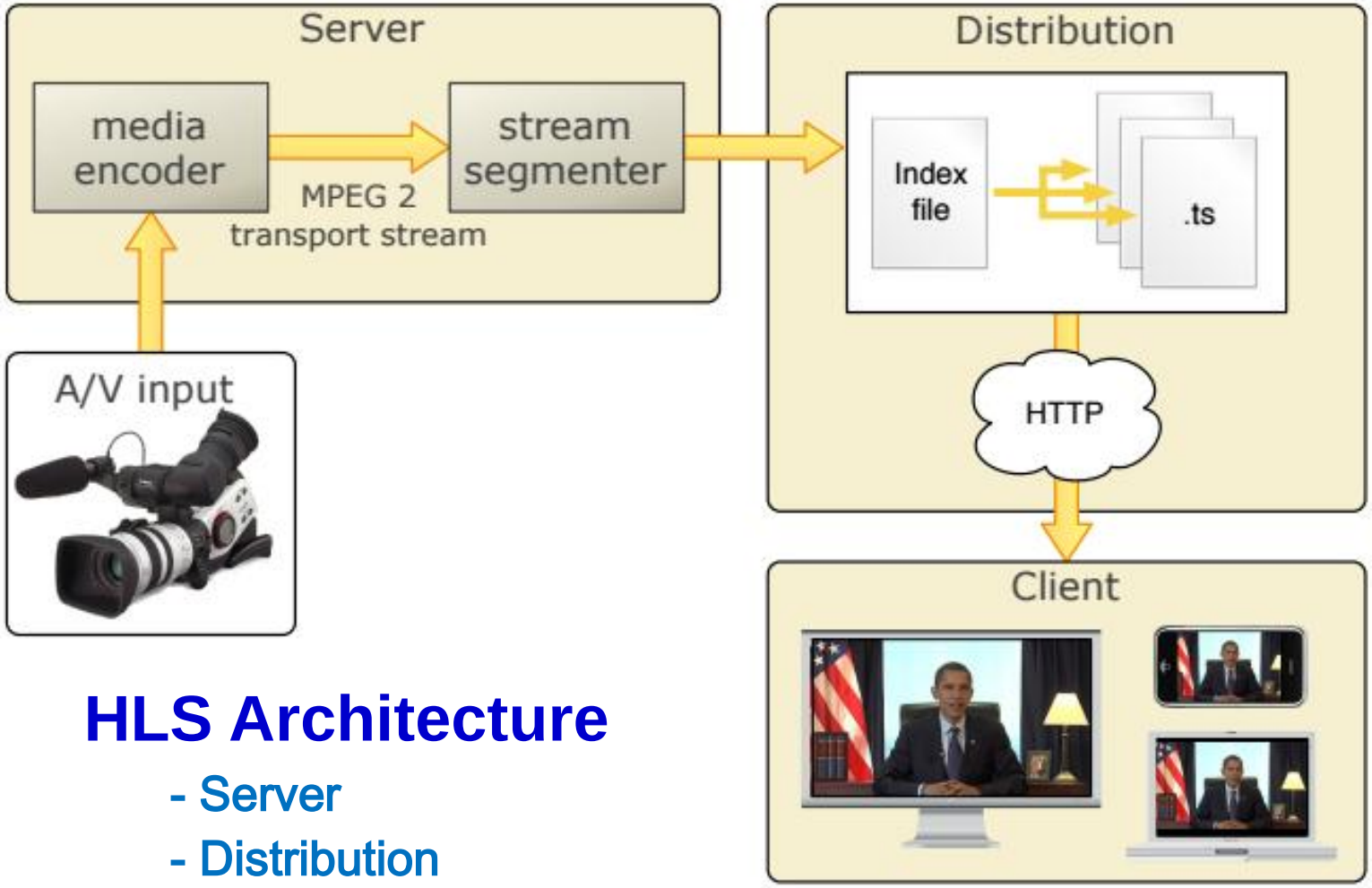
- Streaming audio or video to iOS-based devices — iPhone, iPod touch, iPad, Apple TV, and on desktop computers (Mac OS X).
  - All devices running iOS 3.0 and later have built-in client software for HLS
  - The Safari browser can play HLS streams within a webpage (iPad, PC)
  - Safari launches a full-screen media player for HLS on iOS devices with small screens (iPhone, iPod touch).
  - Apple TV2 and later includes an HTTP Live Streaming client
- No need a special server software (using an ordinary web server)
- Supports both live broadcasts and pre-recorded content (VOD).
- supports multiple alternate streams at different bit rates. Client software can switch streams intelligently as network bandwidth changes.
- provides for media encryption and user authentication over HTTPS

iPhone and iPad apps that send large amounts of audio or video data over cellular networks are required to use HLS.

HTTP Live Streaming (HLS)

- **Two main kinds of streaming protocols**
  - RTP over UDP: - transmission efficiently for real time applications
    (e.g., **IPTV**) - run on port 554 which is often blocked by firewall
  - HTTP over TCP: easy to traverse any firewall or proxy server
    (e.g., **HLS**)
- **How to support streaming over HTTP?**
  - HTTP is a request-response protocol, inherently not for streamming
- **HLS** (HTTP Live Streaming)
  - A HTTP-based streaming protocol
  - Implemented by Apple Inc.
  - Supported by quickTime, iOS/Android, etc.
  - HLS architecture: Server, Distribution, Client software

HTTP Live Streaming (HLS)



HLS Architecture

- Server
- Distribution
- Client

HTTP Live Streaming (HLS) - architecture

1. **Server component -** Encodes input streams in a format for delivery.

The server requires a media encoder and a way to break the encoded media into segments and save them as files for delivery.

- compression format: video: H.264/AVC, audio: AAC, MP3, AC-3
- delivery format: MPEG-2 TS (Transport Stream) for audio-video,
  MPEG ES (Elementary Stream) for audio-only media

- **Media Encoder**
  - takes A/V signals, encodes the media, and encapsulates it for transport.
    - any off-the-shelf encoder can be used (e.g., ffmpeg)
- **MediaStreamSegmenter**
  - A software takes a TS stream from the encoder and divides it into a series of media files (.ts) of equal duration.
    - **MediaFileSegmenter**: If you already have a encoded media file, use this to encapsulate it in an MPEG-2 TS and break it into segments of equal duration.
  - It also creates an index file: **.M3U8** (.ts playlist), or **.m3u** (.mp3 playlists).
- Other tools (by Apple Inc.): VariantPlaylistCreator, StreamValidator

HTTP Live Streaming (HLS) - architecture

2. Distribution component

- Consists of standard **web servers**
  - For large scale distribution, CDN (Content Delivery Networks) can be used.
- The short **.ts files** and **index file (.m3u8)** are all placed on server
- **.m3u8 example**

the client MUST wait for at least the target duration before attempting to reload the Playlist file again

.ts length: 3.998s

4x5=20(s) delay in this example

```
#EXTM3U
#EXT-X-MEDIA-SEQUENCE:4454
#EXT-X-TARGETDURATION:4
#EXTINF:3.998, no desc
25133_src/4460.ts
#EXTINF:3.992, no desc
25133_src/4461.ts
#EXTINF:3.985, no desc
25133_src/4462.ts
#EXTINF:3.979, no desc
25133_src/4463.ts
#EXTINF:3.996, no desc
25133_src/4464.ts
```

```
#EXTM3U
#EXT-X-MEDIA-SEQUENCE:4455
#EXT-X-TARGETDURATION:4
#EXTINF:3.992, no desc
25133_src/4461.ts
#EXTINF:3.985, no desc
25133_src/4462.ts
#EXTINF:3.979, no desc
25133_src/4463.ts
#EXTINF:3.996, no desc
25133_src/4464.ts
#EXTINF:3.998, no desc
25133_src/4465.ts
```

HTTP Live Streaming (HLS) - architecture

- **More examples of index files**

twitch

```
#EXTM3U
#EXT-X-VERSION:3
#EXT-X-TARGETDURATION:5
#ID3-EQUIV-TDTG:2016-11-26T02:40:23
#EXT-X-MEDIA-SEQUENCE:376
#EXT-X-TWITCH-ELAPSED-SYSTEM-SECS:1511.137
#EXT-X-TWITCH-ELAPSED-SECS:1508.980
#EXT-X-TWITCH-TOTAL-SECS:1535.137
#EXTINF:4.000,
index-0000000377-6zCW.ts
#EXTINF:4.000,
index-0000000378-vHZS.ts
#EXTINF:4.000,
index-0000000379-Gkgv.ts
#EXTINF:4.000,
index-0000000380-PNoG.ts
#EXTINF:4.000,
index-0000000381-h58g.ts
#EXTINF:4.000,
index-0000000382-W88t.ts
```

livehouse.in

```
#EXTM3U
#EXT-X-VERSION:3
#EXT-X-ALLOW-CACHE:NO
#EXT-X-MEDIA-SEQUENCE:2291
#EXT-X-TARGETDURATION:6
#EXTINF:5.2090001106262207,
1480116261segment_v02291.ts
#EXTINF:5.2080001831054688,
1480116261segment_v02292.ts
#EXTINF:5.2080001831054688,
1480116261segment_v02293.ts
```

HTTP Live Streaming (HLS) - session

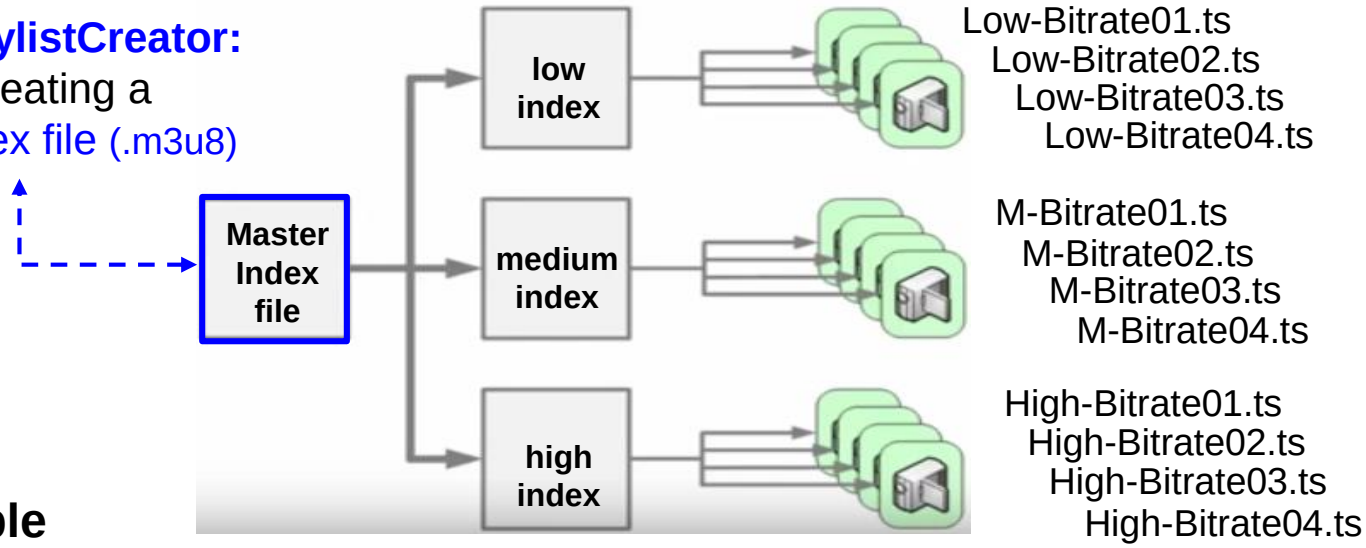
- **HLS supports two types of sessions**
  - **VOD** (video on demand) and **event** (live broadcasting) sessions.
- **VOD sessions**
  - media files are available representing the entire duration of presentation.
  - The index file is static and contains a complete list of all files created since the beginning of the presentation.
  - This kind of session allows the client full access to the entire program.
- **Event sessions**
  - suitable for continuous **live broadcasting**
  - as new media files are created and available, the index file is updated
  - The new index file lists the new media files. Older media files can be discarded and removed from the index file (.m3u8)

    → as a sliding window with a time range the user can seek within.
  - Alternatively, keep listing all the media files in the index file, converting to VOD session after the event completes.

HTTP Live Streaming (HLS) - architecture

- **Support various bitrates and quality - master index file**
  - More than one version of source material is available.

VariantPlaylistCreator:
a tool for creating a
master index file (.m3u8)



- **Example**

```
#EXTM3U
#EXT-X-STREAM-INF:BANDWIDTH=1280000,AVERAGE-BANDWIDTH=1000000
http://example.com/low.m3u8
#EXT-X-STREAM-INF:BANDWIDTH=2560000,AVERAGE-BANDWIDTH=2000000
http://example.com/mid.m3u8
#EXT-X-STREAM-INF:BANDWIDTH=7680000,AVERAGE-BANDWIDTH=6000000
http://example.com/hi.m3u8
```

HTTP Live Streaming (HLS) - architecture

- **Web server configuration**

MIME-type associations for .M3U8 and .ts files.

File Extension	MIME Type
.M3U8	application/x-mpegURL or vnd.apple.mpegURL
.ts	video/MP2T

- **Index file compression**
  - For long index file, you can enable on-the-fly .gzip compression
  - ➔ HLS client automatically unzips the index files.
- **Time-to-live (TTL) for index files** (.m3u8)
  - Live broadcasts: the index file are frequently overwritten, and the latest version should be downloaded for each request ➔ small TTL
  - VOD: the index file is static and downloaded only once ➔ large TTL
- Mediastreamvalidator : a tool for validating HLS streams and servers

HTTP Live Streaming (HLS) - architecture

- **HTML page for HLS**
  - to include the HTML5 <video> tag, using an .M3U8 playlist file as the video source.
  - Example:

```
<html>
<head>
        <title> HLS example </title>
</head>
<body>
        < video src= http://devimages.apple.com/samples/test.m3u8
          height="300" width="400"  >
        </video>
</body>
</html>
```

HTTP Live Streaming (HLS) - architecture

3. Client software

- Downloads, reassembles, and decodes the media for presentation to users.
- Read the index file and then request listed .ts files in order and displays them.
- If a higher-quality stream is available and the bandwidth appears sufficient to support it, the client switches to a higher quality.
- If a lower-quality stream is available and the current bandwidth appears insufficient to support the current stream, the client switches to a lower quality.
- For seamless transitions between alternate streams, the audio portion should be identical in all versions.

Summary

- HLS works by breaking the overall steam into a sequence of small HTTP-based file downloads, each download uses MPEG-TS as its container, H.264 as its video format, and AAC as its audio format.
- To start the streaming, the client downloads a playlist (.m3u8) containing the metadata for the location of various sub-streams (.ts).

HTTP Live Streaming (HLS) - characteristics

- Scalability
  - It works well with HTTP-based CDNs for better scalability
- Support various bitrates and various play quality
  - Client can adjust its play quality dynamically according to network bandwidths.
  - the system must encode video in different qualities and generate an index file that contains the locations of different quality levels (next slide).
- Copyright protection
  - HLS also specifies a standard encryption mechanism using AES and a method of secure key distribution to provide a simple DRM system.
- Trick mode
  - Later versions of HLS protocol also provide for trick mode fast-forward and rewind and integration of subtitles.

Media Streaming

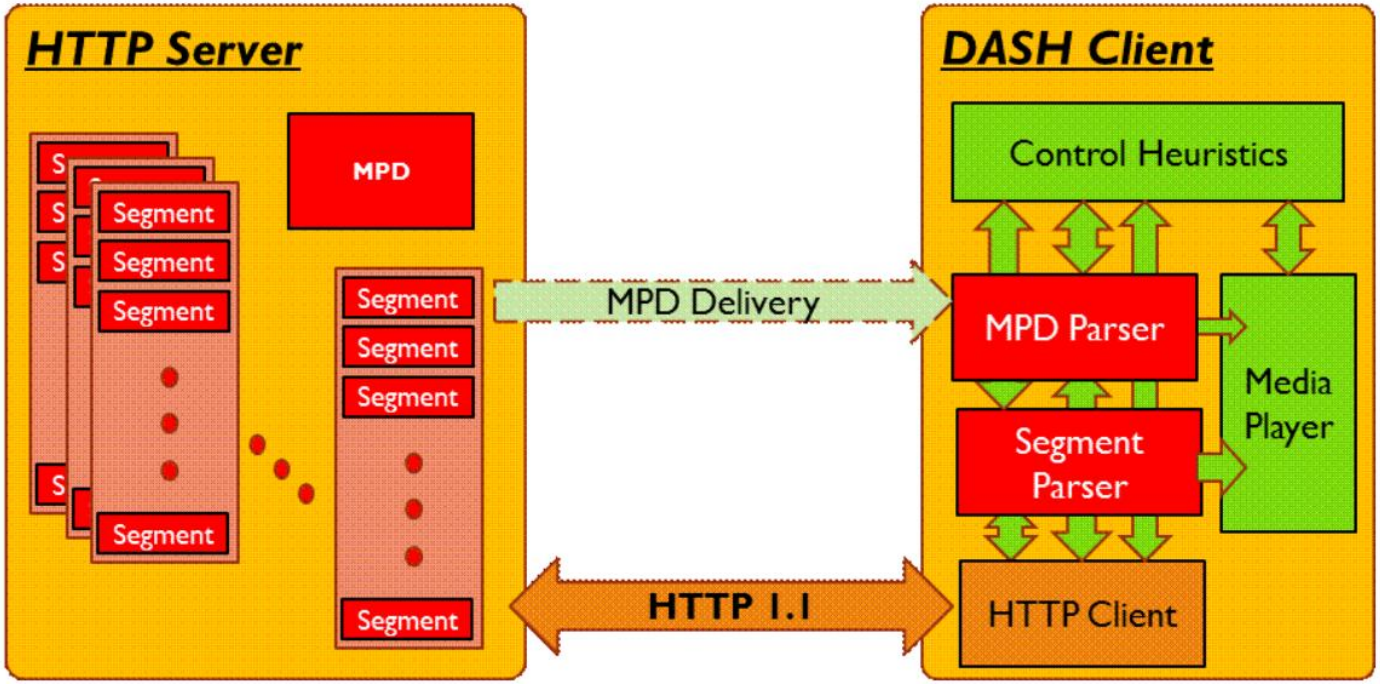
- IPTV v.s. Internet TV
- Media Transport
  - ES, PES, PS, TS
- Streaming Protocol
  - DVB IPTV
  - HTTP Live Streaming (HLS)
  - MPEG-DASH
- Streaming Server Architecture
- P2P Streaming

MPEG-DASH

- The 1st **international standard** for adaptive **streaming over HTTP**, proposed by MPEG
- Deployed on **ordinary Web Server** through HTTP
- **HTML5** supports MPEG-DASH through the use of media source extension → need to use an additional js player (e.g., dash.js)
- **Adapt** to changing network conditions
- **codec-agnostic**: can use any coding format like H.265, H.264, VP9 etc.
- Two profiles: **onDemand** profile, **live** profile
- Support for **multiview**
- Support for efficient **trick** mode

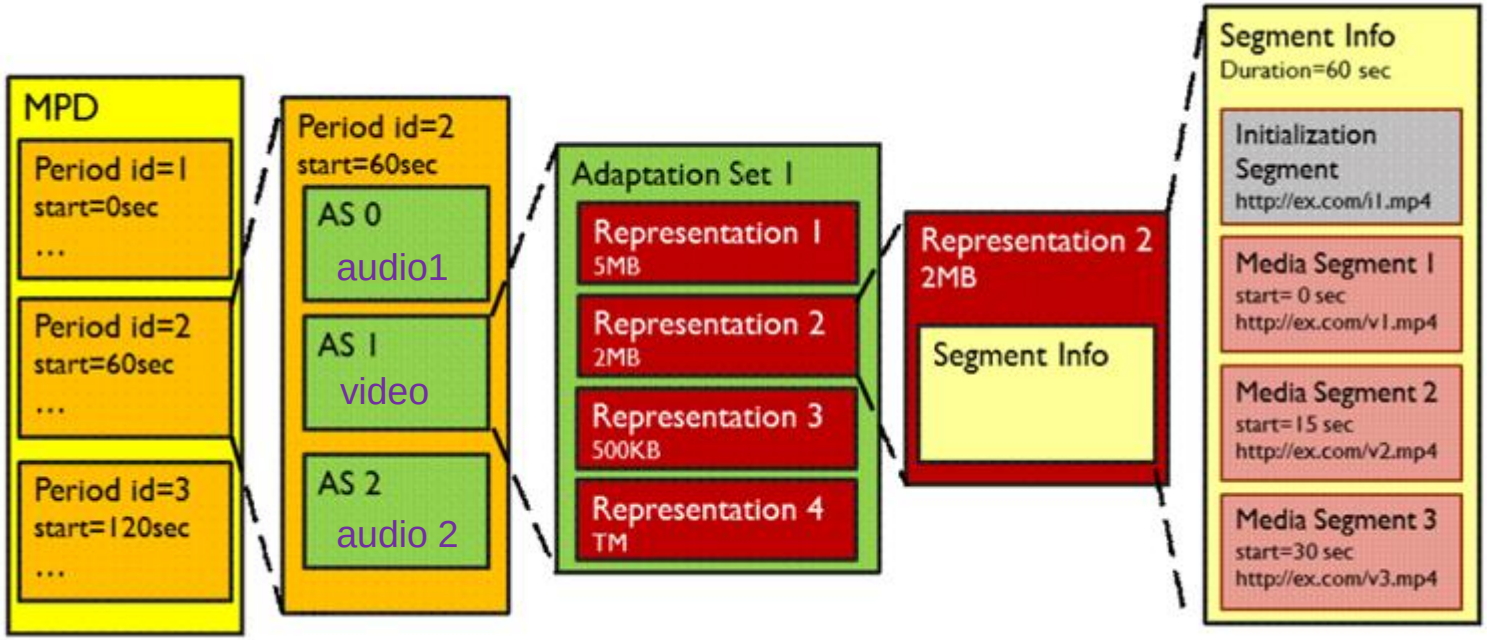
MPEG-DASH

- **How does MPEG-DASH work**
  - It breaks the content into a sequence of small HTTP-based segments.
  - a manifest file (.mpd) that describes these file segments.
  - The content is played back by an MPEG-DASH client.



MPEG-DASH

- **MPD hierarchical data model**
  - **Period**: to specify the starting time and duration
  - **adaptation set**: different AS for different media sources. (e.g., video, audio)
  - **representation**: encoded alternative of the same media, varying from other representations by bitrate, resolution, #(channels), etc.
  - **segment**: a small media file having a URI, can be downloaded by HTTP GET



MPEG-DASH

- **Streaming Generation**
  - To encode/transcode audio/video into **.mp4** (**ISO/IEC 14496-14**)
    - e.g., use **ffmpeg**
    - MP4 (.mp4) is a media container, while MPEG4 is video codec
      - .mp4 = video (mpeg4/h.264/..) + audio (mp3/aac/..) + subtitle
      - Other container for mpeg4 formats: .avi, .mkv, quicktime, etc.
  - **MP4Box:** segments the encoded media into MPEG DASH chunks, and generates initialization segments and the manifest file (.mpd),
    - two profiles by –profile live or –profile onDemand
    - File structure example

```
/segments/news.mp4.mpd ← Manifest file                    .m4s / .mp4 / .dash
/segments/main/
/segments/main/news100 contains (1.m4s,2.m4s,3.m4s ..)
/segments/main/news200 contains (1.m4s,2.m4s,3.m4s ..)
/segments/main/news300 contains (1.m4s,2.m4s,3.m4s ..)   4 bitrates
/segments/main/news400 contains (1.m4s,2.m4s,3.m4s ..)
```

MPEG-DASH .MPD (live profile)

```
<?xml version="1.0"?>
<MPD xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance" xmlns="urn:mpeg:DASH:schema:MPD:2011 “
     xsi:schemaLocation="urn:mpeg:DASH:schema:MPD:2011" profiles="urn:mpeg:dash:profile:isoff-main:2011"
     type="static" mediaPresentationDuration="PT0H9M56.46S">
    <BaseURL> http://example.com/segments </BaseURL>
    <Period start="PT0S">
       <AdaptationSet bitstreamSwitching="true">

       <Representation id="0" codecs="avc1" mimeType="video/mp4" startWithSAP="1" bandwidth="46986">
       <SegmentBase> <Initialization sourceURL="main/news100/1.m4s" range="0-862"/> </SegmentBase>
       <SegmentList duration="1">
           <SegmentURL media="main/news100/2.m4s" mediaRange="863-7113"/>
           <SegmentURL media="main/news100/3.m4s" mediaRange="7114-14104"/>
           <SegmentURL media="main/news100/4.m4s" mediaRange="14105-17990"/>
       </SegmentList> </Representation>

       <Representation id="1" codecs="avc1" mimeType="video/mp4" startWithSAP="1" bandwidth="91932">
       <SegmentBase> <Initialization sourceURL="main/news200/1.m4s" range="0-864"/> </SegmentBase>
       <SegmentList duration="1">
           <SegmentURL media="main/news200/2.m4s" mediaRange="865-11523"/>
           <SegmentURL media="main/news200/3.m4s" mediaRange="11524-25621"/>
           <SegmentURL media="main/news200/4.m4s" mediaRange="25622-33693"/>
       </SegmentList> </Representation>

            ...

       </AdaptationSet>
    </Period>
</MPD>
```

MPEG-DASH .MPD (onDemand profile)

```xml
<?xml version="1.0" encoding="UTF-8"?>
<MPD xmlns:xsi=http://www.w3.org/2001/XMLSchema-instance
xmlns="urn:mpeg:dash:schema:mpd:2011"
xsi:schemaLocation="urn:mpeg:dash:schema:mpd:2011 DASH-MPD.xsd"
type="static"
mediaPresentationDuration="PT654S"
minBufferTime="PT2S"
profiles="urn:mpeg:dash:profile:isoff-on-demand:2011">
<BaseURL>http://example.com/ondemand/</BaseURL>
   <Period>
     <AdaptationSet mimeType="audio/mp4" codecs="mp4a.40.5" subsegmentAlignment="true">
          <Representation id="1" bandwidth="64000">
          <BaseURL>ElephantsDream_AAC48K_064.mp4.dash</BaseURL>
          </Representation>
     </AdaptationSet>
     <AdaptationSet mimeType="video/mp4" codecs="avc1.42401E" subsegmentAlignment="true">
          <Representation id="2" bandwidth="100000" width="480" height="360">
               <BaseURL>ElephantsDream_H264BPL30_0100.264.dash</BaseURL>
          </Representation>
          <Representation id="3" bandwidth="175000" width="480" height="360">
               <BaseURL>ElephantsDream_H264BPL30_0175.264.dash </BaseURL>
          </Representation>
          <Representation id="4" bandwidth="250000" width="480" height="360">
               <BaseURL>ElephantsDream_H264BPL30_0250.264.dash</BaseURL>
          </Representation>
          <Representation id="5" bandwidth="500000" width="480" height="360">
               <BaseURL>ElephantsDream_H264BPL30_0500.264.dash</BaseURL>
          </Representation>
     </AdaptationSet>
    </Period>
</MPD>
```

MPEG-DASH

- **HTML page for MPEG-DASH**
  - Once you have generated MPD file, you can reference it in HTML like

```
<video src="my.mpd" type="application/dash+xml">
</video>
```

  - It might be wise to provide a fallback

```
<video>
      <source src="my.mpd" type="application/dash+xml">
      <!-- fallback -->
      <source src="my.mp4" type="video/mp4">
      <source src="my.webm" type="video/webm">
</video>
```

  - MPEG-DASH playback relies on dash.js and browser support for MSE (Media Source Extension).
  - To add dash.js player, you can

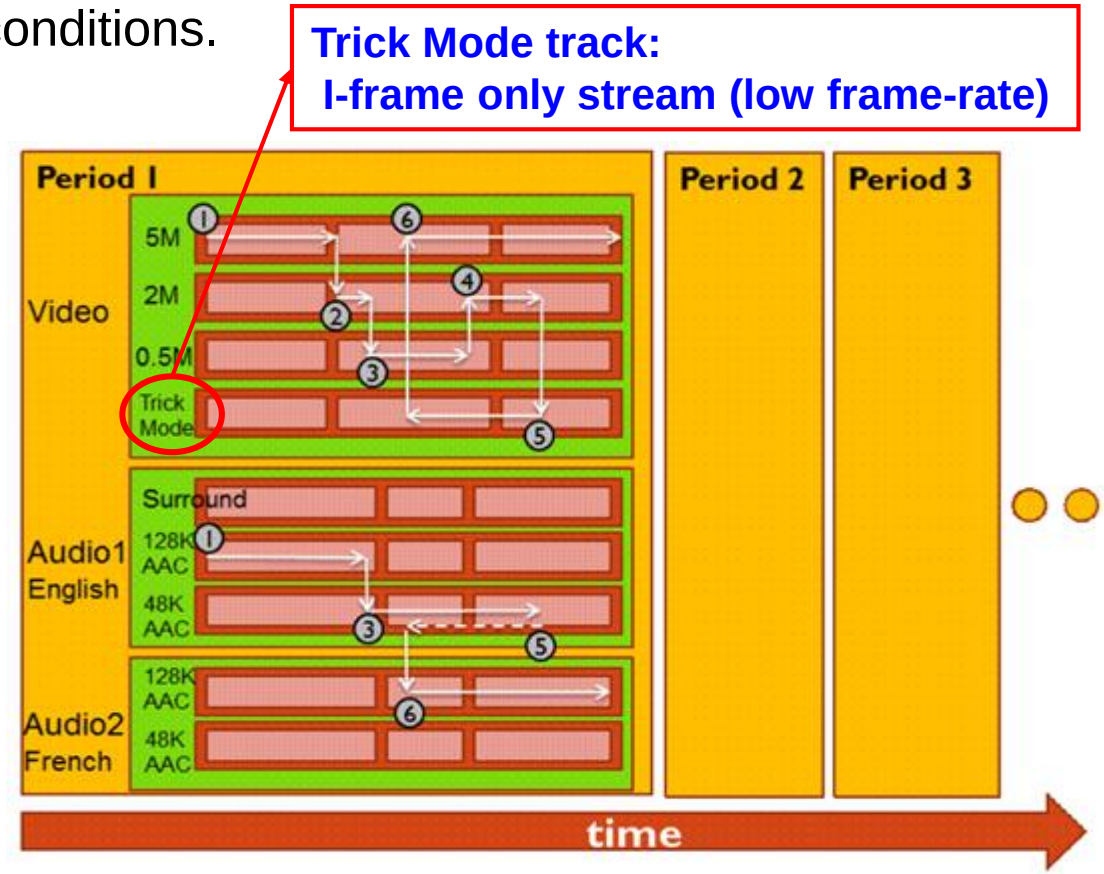
```
<script src=”js/dash.all.js”></script>
```

MPEG-DASH

- **Adaptive Streaming**
  - The content is encoded at a variety of different bit rates
  - a manifest file (.mpd) that describes these versions.
  - The client decides which version of the segments to download/play based on network conditions.

- **A Simple Case**

2: bandwidth insufficient
3: bandwidth reduced
4. bandwidth increased
5: user pause and rewind
6: tune to French audio



Streaming Protocols – over HTTP

- Benefits of streaming over HTTP
  - Internet infrastructure efficiently support HTTP (e.g., Supporting a large number of streaming can be managed by CDNs)
  - HTTP is firewall friendly: almost all firewalls support its outgoing links.
  - No need for clients to maintain a session on server → reduce overhead.
- Well-known streaming protocols (over HTTP)
  - **HTTP Dynamic Streaming (HDS) - Adobe**
    - Flash Media Streaming Server is proprietary and Flash Player (client)
    - Used by YouTube for on-demand streaming
  - **Microsoft Smooth Streaming (MSS) – Microsoft Inc.**
    - Microsoft Windows Server. Microsoft Silverlight (client)
  - **HTTP Live Streaming (HLS) – Apple Inc.**
  - **Dynamic Adaptive Streaming over HTTP (DASH) – MPEG**

Both use ordinary HTTP server and browser

Streaming Protocol - Comparison

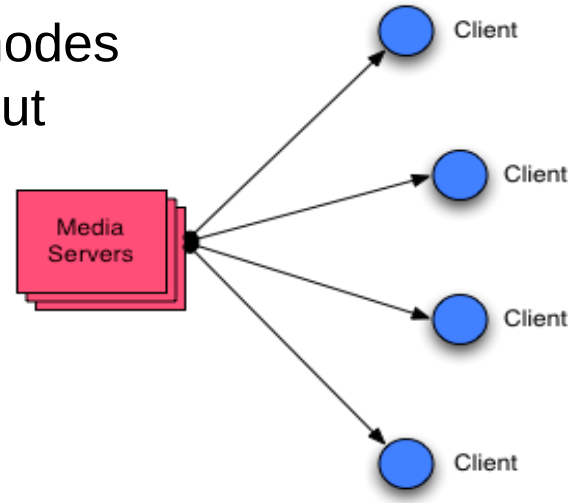
features	Adobe HDS	Apple HLS	MS MMS	MPEG DASH
Using Ordinary HTTP Servers		☺		☺
Official International Standard				☺
Multiple Audio Channels		☺	☺	☺
Content Protection (DRM)	☺	☺	☺	☺
Closed Captions / Subtitles	☺	☺	☺	☺
Efficient Ad Insertion				☺
Fast Channel Switching	☺		☺	☺
Support’s multiple CDNs				☺
HTML5 support				☺
HEVC ready (UHD/4K)				☺
Agnostic to video codecs				☺
ISO Base Media File Format	☺		☺	☺
MPEG-2 TS		☺		☺
Multiple Video Views				☺
Efficient Trick Modes				☺

IPTV

- What's IPTV?
- Media Transport
- Streaming Protocol
- Streaming Server Architecture

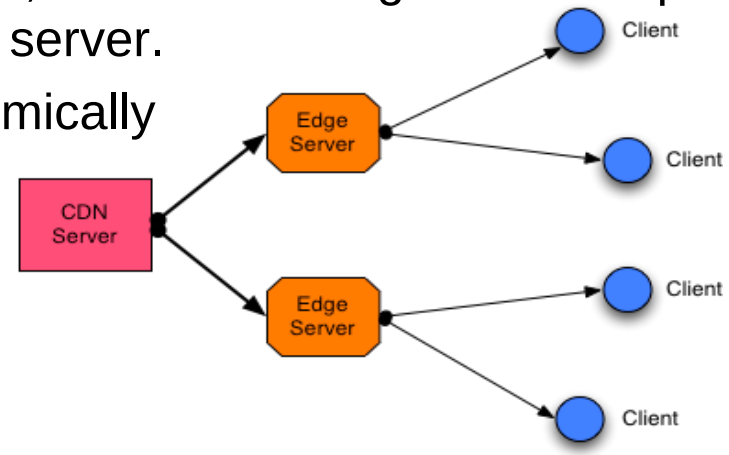
Streaming Server Architecture

- **Client/Server Network**
  - A Client/Server network is a distributed network which consists of a higher performance server(s) and several mostly lower performance clients
  - The server is the only provider of content and service
  - A client only requests content of the execution of services, without sharing any of its own resource.
  - Within C/S network, the participating nodes can either act as a server or a client, but cannot embrace both capabilities.



Streaming Server Architecture

- **Content Delivery Network (CDN)**
  - **A system of computers (called edge servers) networked together across the Internet that cooperate to deliver content to end users**
    - end-users are now served by edge servers instead of the content source server (CDN server).
    - improve performance, scalability, and cost efficiency.
  - **Modern CDNs can dynamically distribute contents to strategically placed edge servers to optimize the performance**
    - Edge servers should provide views of loadings and statistics, reveal which contents are popular, show active regions and report exact viewing details to the CDN server.
    - Requests for content are algorithmically directed to nodes that are optimal in some way.



Streaming Server Architecture

- **Peer-to-Peer Network (P2P)**
  - P2P is a way of structuring distributed applications such that individual nodes have symmetric roles. Rather than being divided into clients and servers each with distinct roles (such Web clients vs. Web servers), in P2P system a node may act as both a client and a server.
  - A distributed network architecture may be called a P2P network, if the participants share a part of their own hardware resources (processing power, storage capacity, network link capacity, printer, …)
  - Peers have equivalent responsibility: resource providers and users.
  - Create and maintain networks without central entities.
- **Why P2P?**
  - Get rid of servers – single point of failure, centralized control and management, reduced access fee and management fee, …
  - Clients are not so dumb – billions of Mhz CPU, tons of terabytes disk, millions of gigabits network bandwidth, …